CS319: Scientific Computing

Projects; A `Vector` class; Linear Equation Solving

Dr Niall Madden

Week 9: 11+13 March, 2026

Slides and examples:
https://www.niallmadden.ie/2526-CS319

2526-CS319

0. Outline

1. Projects!
2. Review of classes
3. Vectors and Matrices
4. A vector `class` (Basic version)
   - C++ "Project"
   - Adding two vectors
5. Systems of equations
   - Algorithms
   - Asides...
6. Iterative methods
7. Jacobi's Method
   - Lab 7
8. Gauss-Seidel Method
   - More history
9. Matrix-based Jacobi
10. Triangular systems
11. Matrix-based Gauss-Seidel
12. Some further details

1. Projects!

Notes for this part are at:
https://www.niallmadden.ie/2526-CS319/2526-CS319-Projects.pdf

2. Review of classes

class

In C++, we define new class with the `class` keyword.
An instance of the class is called an "*object*".
A `class` combines by data and functions (called "methods").

Within a class, code and data may be either

- **Private**: accessible only to another part of that object, or
- **Public**: other parts of the program can access it.

Roughly,

- keep data elements `private`,
- make function elements `public`.

2. Review of classes

The basic syntax for defining a class:

```
class class-name {
private:
    ...     // private functions and variables
public:
    ...     // public functions and variables
};
```

class-name becomes a new object type—one can now declare objects to be of type *class-name*.

This is only a declaration. Therefore,

- functions are not defined, though the prototype is given,
- variables are declared but are not initialised,
- the declaration block is delineated by { and }, and terminated with a semicolon.
- *scope resolution operator*, `::`, used in function definition.

2. Review of classes

- A **Constructor** is a public method of a class, that has the same name as the class. It's return type is not specified explicitly. It is executed whenever a new instance of that class is created.
- A **destructor** is a method that is called on an object whenever it goes out of scope. The name of the destructor is the same as the class, but preceded by a tilde.

3. Vectors and Matrices

This is a course in Scientific Computing.

Many advanced and general problems in Scientific Computing are based around **vectors** and **matrices**. So one of our goals is to implement C++ classes for such structures, along with standard operations such as matrix-vector multiplication.

Along the way, we'll learn about

- operator overloading;
- `friend` functions and the `this` pointer;
- static variables.
- and much more

Our first step will be to study some problems and applications so that, before we design any classes or algorithms, we'll know what we will use them for. These problems include:

1. Basic analysis of matrices, for example with applications to image processing, graphs and networks.
2. Solution of linear systems of equations, for example with applications to data fitting;
3. Estimation of (certain) eigenvalues, for example with applications to Network Science.

Of these problems, probably the most ubiquitous is the solution of (large) systems of simultaneous equations.

That is, we want to solve a linear system of 3 equations in 3 unknowns: *find* x_1, x_2, x_3, *such that*

$$\begin{aligned} 3x_1 + 2x_2 + 4x_3 &= 19 \\ x_1 + 2x_2 + 3x_3 &= 14 \\ 5x_1 + 1x_2 + 6x_3 &= 25 \end{aligned}$$

This can be expressed as a **matrix-vector equation:**

$$\begin{pmatrix} 3 & 2 & 4 \\ 1 & 2 & 3 \\ 5 & 1 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 19 \\ 14 \\ 25 \end{pmatrix}$$

More generally, the linear system of N equations in N unknowns: *find* $x_1, x_2, \ldots, x_N$, *such that*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1N}x_N &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2N}x_N &= b_2 \\ &\vdots \\ a_{N1}x_1 + a_{N2}x_2 + \cdots + a_{NN}x_N &= b_N. \end{aligned}$$

This, as a **matrix-vector equation** is:

$$\underbrace{\begin{pmatrix} a_{11} & a_{12} & \ldots & a_{1N} \\ a_{21} & a_{22} & \ldots & a_{2N} \\ \vdots & & \ddots & \vdots \\ a_{N1} & a_{N2} & \ldots & a_{NN} \end{pmatrix}}_{\text{Matrix}} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix}}_{\text{Vector}} = \underbrace{\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix}}_{\text{Vector.}}$$

So, to proceed, we need to be able to represent **vectors** and **matrices** in our codes.

Our first focus will be on defining a class of vectors. This version will be quite **basic** (hence the file names). Will be developed later.

Intuitively, we know it needs the following components:

Data	Method
Entries (as doubles)	Add two vectors.
Number of Entries (unsigned int.	Scalar - vector multiplication.
	dot product.
	set & get values.
	(etc ... eg, initialize, compute norms ...)

4. A vector class (Basic version) C++ "Project"

Due to the level of detail in the matrix and vector classes, the following example is divided into three source files:

1. `VectorBasic.h`, the header file which contains the class definition. Include this header file in another source file with:
   `#include "VectorBasic.h"`
   Note that this is **not** `<VectorBasic.h>`
2. `VectorBasic.cpp`, which includes the code for the methods in the *`Vector`* class;
3. `OOTestVectorBasic.cpp`, a test stub.

In whatever compiler you are using, you'll need to create a **project** that contains all the files. (Ask Niall for help if needed).

See VectorBasic.h for more details

```
// File: VectorBasic.h (simple version)
// Author: Niall Madden ¡Niall.Madden@UniversityOfGalway.ie¿
// Date: Week 9 of CS319
// What: Header file for vector class
// See also: VectorBasic.cpp and 01TestVectorBasic.cpp
class Vector {
private:
  double *entries;
  unsigned int N; // number of Entries "length".
public:
  Vector(unsigned int Size=2); Constructor
  ~Vector(void); destructor

  unsigned int size(void) {return N;}; // in-line function.
  double geti(unsigned int i);
  void seti(unsigned int i, double x);

  void print(void);
  double norm(void); // Compute the 2-norm of a vector
  void zero(void); // Set entries of vector to zero.
};
```

VectorBasic.cpp

```
Vector::Vector(unsigned int Size)
{
  N = Size;
  entries = new double[Size];
}

Vector::~Vector()
{
  delete [] entries;
}

void Vector::seti(unsigned int i, double x)
{
  if (i<N)
    entries[i]=x;
  else
    std::cerr << "Vector::seti(): Index out of bounds."
              << std::endl;
}
```

Constructor

Destructor.

VectorBasic.cpp continued

```
double Vector::geti(unsigned int i)
{
  if (i<N)
    return(entries[i]);
  else {
    std::cerr << "Vector::geti(): Index out of bounds."
              << std::endl;
    return(0);
  }
}

void Vector::print(void)
{
  for (unsigned int i=0; i<N; i++)
    std::cout << "[" << entries[i] << "]" << std::endl;
}
```

VectorBasic.cpp continued

```
double Vector::norm(void)
{
  double x=0;
  for (unsigned int i=0; i<N; i++)
    x+=entries[i]*entries[i];
  return (sqrt(x));
}

void Vector::zero(void)
{
  for (unsigned int i=0; i<N; i++)
    entries[i]=0;
}
```

$$\sqrt{\sum_i (v_i)^2}$$

4. A vector class (Basic version) Adding two vectors

Here is a simple implementation of a function that computes
$c = \alpha a + \beta b$

pass by reference

See 00TestVectorBasic.cpp for more details

```
// c = alpha*a + beta*b where a,b are vectors; alpha, beta are scalars
void VecAdd (vector &c, vector &a, vector &b,
        double alpha, double beta)
{
  unsigned int N;
  N = a.size();

  if ( (N != b.size()) )
    std::cerr << "dimension mismatch in VecAdd " << std::endl;
  else
  {
    for (unsigned int i=0; i<N; i++)
        c.seti(i, alpha*a.geti(i)+beta*b.geti(i) );
  }
}
```

Compile with: g++ 00TestVectorBasic.cpp VectorBasic.cpp

Finished here on Wednesday